

SHL Conversational Assessment Recommender, Approach

Live API: <https://shl-recommender-m045.onrender.com> · **Demo:** <https://web-eight-theta-60.vercel.app> · **Repo:** <https://github.com/ghostiee-11/shl-recommender>

Problem framing

Recruiters describe roles in their own words; the SHL catalog speaks in instrument names. The agent's job is to translate, in dialogue, with grounded references to the live catalog. Four behaviors required: clarify, recommend (1-10 items), refine, compare. Plus refusal of off-topic and prompt-injection inputs. Stateless `POST /chat`, ≤ 8 turns, ≤ 30 s per call, schema-locked response.

Architecture

```
POST /chat (stateless) → FastAPI → Orchestrator
                                |
                                |— refusal (deterministic + LLM tiebreak)
                                |— compare detector (regex + 3-stage fuzzy lookup)
                                |— slot extractor (1 LLM call, evidence-tagged)
                                |— decision policy (pure function, table-driven)
                                |— hybrid retrieval (BM25 + dense + RRF)
                                |— LLM reranker (with safe fallback)
                                |— reply composer (LLM prose, deterministic fallback)
```

Three LLM calls per recommend turn (extract, rerank, compose); one or zero on the other paths. Every LLM site has a deterministic fallback so the agent never returns junk under provider failure.

Retrieval

Hybrid: BM25 + dense (text-embedding-3-small) + RRF (k=60). BM25 catches exact instrument names like `0PQ32r`; dense catches paraphrase ("senior leadership selection" → leadership reports). RRF is parameter-free, so the system doesn't overfit to the 10 public traces. Catalog vectors are pre-computed once and shipped as `data/catalog_embeddings.npy`; query vectors come from the OpenAI API per turn (~150 ms TTFT). After fusion, an optional metadata filter restricts by primary `test_type`. The LLM reranker takes the top 20 hybrid hits and reorders given the structured query; on any LLM failure it falls back to hybrid order so the recommendation list stays grounded.

Agent design

Stateless reconstruction via an embedded HTML-comment hint (`<!--state:{...}-->`) appended to every assistant reply. Spec-compliant string `reply`; the next turn parses the hint to recover slots, prior shortlist, asked-slots set, and turn index. Falls back to LLM re-extraction over the full history if the hint is missing or malformed.

Decision policy is a pure function of (`slots`, `intent`, `turn_index`, `has_prior_shortlist`). Refuse > compare > refine > recommend > clarify. Two safety nets that earned their keep in testing: - **JD-blob heuristic**, when the slot extractor returns nothing useful (rate limit, schema slip) but the user message contains role keywords + length, treat as a brief instead of asking another clarifying question. - **Bias-to-commit**, after one clarification, if the user gave us role + (skills or seniority), recommend rather than asking again. Recruiters dislike interrogations.

Refusal layer is deterministic for hard patterns (prompt injection, role override). Soft patterns (salary, legal, HR advice) consult an LLM tiebreak and conservatively refuse on failure. Templates are string constants, not LLM-generated, so they cannot be jailbroken.

Compare path is also deterministic for detection: regex matches "compare X and Y", three-stage fuzzy lookup (exact → substring → token coverage with stopword filtering) finds catalog items, the composer narrates the diff. Around 3 s end-to-end, no slot-extraction call.

Prompts

Versioned and centralised in `agent/prompts.py`. JSON-mode discipline for the slot extractor and reranker (Pydantic-validated, falls back on schema violation). The composer runs in plain-prose mode (JSON tone makes replies feel formal) with a hard rule against em/en dashes; a post-processor strips any that slip through anyway.

Evaluation

`eval/run_full.py` runs two arms: **10 behavior probes** (refusal, schema, URL grounding, length bound, refinement honored, compare grounded, no-recommend-on-injection, etc.) and **trace replay** over the 10 reference dialogues with Recall@10 and nDCG@10 metrics, bootstrapped 95 % CIs in the report. Plus `eval/sweep_conversations.py` runs 10 representative recruiter scenarios end-to-end against the deployed URL with binary assertions.

Live results against the deployed Render service: - `/health` 200 in 12 ms · `/chat` ~5-10 s warm · 200 schema-compliant for every turn observed - **14 / 14 conversational sweep assertions pass** against <https://shl->

What didn't work (and how I measured)

1. **Symmetric query expansion hurt by 4 pp** (Recall@10 0.504 → 0.464). Synonyms diluted the dense bi-encoder which already captures semantic similarity. Switched to **asymmetric** expansion: synonyms feed BM25 only, dense gets the clean query. Recovered the loss with the architectural win for holdout robustness.
2. **MS-MARCO cross-encoder rerank hurt by 9 pp** (0.504 → 0.417). Distribution mismatch: the model was trained on web-search query/passage pairs, but our queries are conversational hiring intent and our docs are structured product specs. Kept the implementation in-tree as opt-in, routed the rerank slot to the LLM reranker.
3. **bge-small + sentence-transformers OOM'd Render free tier**. Local dev hid this (no memory ceiling). Pivoted to OpenAI `text-embedding-3-small` with catalog vectors pre-computed offline; runtime image dropped from ~5 GB to ~200 MB and cold start from ~30 s to ~3 s.
4. **State-hint regex was non-greedy** (`{.*?}`) and broke on nested JSON braces in the embedded state, so refine-path tests failed silently. Caught by the 10-scenario sweep, fixed to greedy `{.*}` anchored by `-->`.
5. **Slot extractor's strict schema** rejected `null` lists that the LLM emits for "no preference". Added a Pydantic `field_validator(mode="before")` that coerces `None` → `[]`.
6. **Free-tier LLM rate limits cascaded** (Groq 30 RPM + Gemini 5 RPM both throttled together during heavy eval runs). Built a per-provider token-bucket throttle inside the LLM router so the agent self-paces instead of cascading 429s.

AI-tool usage

Claude Code assisted with scaffolding and rough planning. The design choices, debugging, eval harness, and most of the code are mine.

Stack & deploy

- **Backend:** Python 3.12, FastAPI, Pydantic v2, FAISS, rank-bm25, openai, groq, google-genai, structlog, tenacity. Tests via pytest (173 green, 5.5 s total).
- **Frontend:** Vite + React + TypeScript + Tailwind v4. Single-page, editorial design.
- **LLM router:** OpenAI primary (Tier-2, 5000 RPM), Groq fallback, Gemini second fallback, with circuit breaker + per-provider token-bucket throttle.
- **Hosting:** Backend on Render free tier (Docker, multi-stage, non-root, ~200 MB image). Frontend on Vercel. Keep-alive cron via GitHub Actions every 14 min so Render never sleeps.